Use **Generics** em Java sempre que quiser garantir segurança de tipos em tempo de compilação, evitar conversões manuais (casts) e criar classes, interfaces ou métodos reutilizáveis para diferentes tipos de dados. [1, 2, 3]

Quando e por que usar Generics

- **Segurança de Tipos (Type Safety):** Quando você deseja garantir que uma estrutura (como uma lista ou uma classe de repositório) aceite apenas um tipo específico de objeto, evitando erros como `ClassCastException` em tempo de execução. [1, 2]
- **Eliminação de Casts:** Quando você quer recuperar elementos de uma coleção (por exemplo, `List<String>`) sem precisar fazer conversões explícitas do tipo `(String) objeto` a toda hora. [1, 2]
- **Reutilização de Código:** Quando você cria uma classe utilitária, um algoritmo ou uma estrutura de dados (como uma fila, pilha ou caixa genérica) que precisa funcionar de forma idêntica para `Integer`, `String`, `Double` ou qualquer objeto personalizado. [1, 2]
- **Trabalho com Coleções:** Sempre que utilizar a Java Collections API (como `List`, `Set`, `Map`), para deixar claro qual é o tipo dos dados armazenados. [1]

Principais Benefícios

- **Detecção precoce de erros:** O compilador avisa se você tentar colocar um tipo errado (por quebrar a regra de `List<Integer>`, por exemplo) antes de rodar o programa.
- **Código mais limpo e legível:** Dispensa a poluição visual de múltiplos `casts` e evita a duplicação de classes para diferentes tipos. [1, 2, 3]

Aqui está um exemplo prático mostrando a diferença entre criar uma **classe genérica** (para reutilizar código) e usar **coringas** (wildcards) com os limites `extends` e `super` (para dar flexibilidade aos métodos).

1. Criando uma Classe Genérica Personalizada

Imagine uma caixa simples que pode guardar qualquer tipo de objeto. Em vez de criar uma `CaixaString`, `CaixaInteger`, etc., você cria apenas uma `Caixa<T>`:

java

```
// O 'T' é um marcador que representa o tipo que será definido ao criar o
objeto
```

```
public class Caixa<T> {
    private T conteudo;

    public void guardar(T conteudo) {
        this.conteudo = conteudo;
    }

    public T pegar() {
        return conteudo;
    }
}
```

Use o código com cuidado.

Como usar no código:

java

```
// Definindo que esta caixa guarda apenas Strings
Caixa<String> caixaDeTexto = new Caixa<>();
caixaDeTexto.guardar("Olá, Generics!");
String texto = caixaDeTexto.pegar(); // Sem necessidade de fazer cast manual!

// A mesma classe agora guardando Inteiros
Caixa<Integer> caixaDeNumero = new Caixa<>();
caixaDeNumero.guardar(123);
Integer numero = caixaDeNumero.pegar();
```

Use o código com cuidado.

---

2. Entendendo Coringas (Wildcards) e Limites

Coringas (`?`) entram em cena quando você precisa que um método aceite listas de tipos parecidos ou relacionados por herança.

Considere esta hierarquia simples:

- `Animal` (Classe pai)
  - `Cachorro` (Filho de Animal)
  - `Gato` (Filho de Animal)

O Limite Superior: **`<? extends T>`** (Produtor / Leitura)

Use `<? extends T>` quando você quer **ler** dados de uma coleção que aceita `T` ou qualquer classe filha de `T`.

java

```java
public class Clinica {
    // Aceita List<Animal>, List<Cachorro>, List<Gato>...
    public static void examinarAnimais(List<? extends Animal> animais) {
        for (Animal animal : animais) {
            // Seguro para leitura: temos certeza de que qualquer item é um Animal
            animal.fazerBarulho();
        }
        // animais.add(new Cachorro()); // ERRO DE COMPILAÇÃO! Não é seguro adicionar itens aqui.
    }
}
```

Use o código com cuidado.

O Limite Inferior: `<? super T>` (Consumidor / Escrita)

Use `<? super T>` quando você quer **escrever/adicionar** dados em uma coleção que aceita `T` ou qualquer classe que seja pai (superclasse) de `T`.

java

```java
public class Canil {
    // Aceita List<Cachorro>, List<Animal>, List<Object>
    public static void adicionarCachorros(List<? super Cachorro> listaDeCachorros) {
        // Seguro para escrita: podemos adicionar um Cachorro sem quebrar o tipo da lista
        listaDeCachorros.add(new Cachorro());

        // Object animal = listaDeCachorros.get(0); // Leitura só retorna Object, pois não sabemos o tipo exato
    }
}
```

Use o código com cuidado.

Regra de Ouro: **PECS**

Para nunca mais esquecer quando usar cada coringa, lembre-se da sigla **PECS**:

- **P**roducer **E**xtends: Se a coleção vai apenas **produzir** dados para o seu código (você vai ler dela), use `? extends T`.
- **C**onsumer **S**uper: Se a coleção vai **consumir** dados do seu código (você vai escrever nela), use `? super T`.

Quer testar esses conceitos criando uma **função prática** ou prefere entender como o compilador do Java apaga esses tipos por baixo dos panos (**Type Erasure**)?

então a existencia de uma classe generics é guardar ou manipular objetos? qual a diferença para uma classe comun?

A existência de uma classe genérica serve para **ambas as coisas: guardar e manipular objetos**. No entanto, a principal diferença para uma classe comum não está no *que* ela faz, mas sim em **como ela garante a segurança dos dados que entram e saem**.

A grande diferença prática é que **uma classe genérica é um modelo abstrato de tipos**, enquanto uma classe comum trabalha com tipos fixos.

A diferença na prática: Classe Comum vs. Classe Genérica

Imagine que você precisa criar uma estrutura simples para segurar um objeto.

1. A abordagem com Classe Comum (Usando `Object`)

Antes do Java 5, para fazer uma classe que aceitasse qualquer objeto, usava-se a classe mãe de todas: `Object`.

java

```
public class CaixaComum {
    private Object conteudo;

    public void guardar(Object conteudo) { this.conteudo = conteudo; }
    public Object pegar() { return this.conteudo; }
}
```

Use o código com cuidado.

- **O problema:** Ela aceita qualquer coisa, mas perde o controle do que está lá dentro.

java

```
CaixaComum caixa = new CaixaComum();
caixa.guardar("Texto"); // Aceita String

// Na hora de pegar, o Java não sabe o que tem dentro. Você é obrigado a fazer
um Cast:
String texto = (String) caixa.pegar();

// Se você esquecer o que colocou e tentar transformar em número:
```

```
Integer numero = (Integer) caixa.pegar(); // ClassCastException (O programa quebra em execução!)
```

Use o código com cuidado.

2. A abordagem com Classe Genérica (**<T>**)

A classe genérica resolve isso criando um "contrato de tipo" no momento em que você instancia o objeto.

java

```
public class CaixaGenerica<T> {
    private T conteudo;

    public void guardar(T conteudo) { this.conteudo = conteudo; }
    public T pegar() { return this.conteudo; }
}
```

Use o código com cuidado.

- **A solução:** O compilador agora vigia o seu código.

java

```
CaixaGenerica<String> caixa = new CaixaGenerica<>();
caixa.guardar("Texto");

String texto = caixa.pegar(); // Não precisa de cast! O Java já sabe que é String.

// caixa.guardar(123); // ERRO DE COMPILAÇÃO! O Java nem deixa você rodar o código.
```

Use o código com cuidado.

Resumo das Diferenças

Característica	**Classe Comum (Fixo ou `Object`)**	**Classe Genérica (<T>)**
Flexibilidade de Tipos	Baixa (tipo único) ou Perigosa (usando `Object`).	Alta (funciona com qualquer t escolher).
Checagem de Erros	Acontece em tempo de **execução** (o app trava na mão	Acontece em tempo de **comp** te avisa antes de rodar).

	do usuário).	
Legibilidade	Poluída com conversões manuais de tipo (`(String)` , `(Caminhao)`).	Limpa, o Java faz a inferência automaticamente.

Então, quando eu mudo de uma classe comum para uma genérica?

Você usa uma classe comum quando a sua lógica de negócio pertence estritamente a um cenário fixo (ex: uma classe `Usuario` só manipula dados de usuário).

Você muda para uma **classe genérica** quando percebe que a estrutura da classe ou os métodos de manipulação seriam exatamente iguais para tipos diferentes (ex: repositórios de banco de dados, filas, caches, listas ou conversores de formato).

O uso de `extends` em parâmetros genéricos permite limitar o tipo que uma classe ou método pode aceitar.